# Amazon EKS Advanced Technical Guide

EKS Auto Mode | Authentication Modes | Endpoint Access | Node IP Sizing

## 1. EKS Auto Mode

**EKS Auto Mode** is a capability provided by AWS to fully automate Kubernetes infrastructure management. It extends AWS's responsibility beyond managing the Control Plane to completely handle operational components of Worker Nodes, Networking, and Storage.

### Key Operational Responsibilities of EKS Auto Mode

- **Compute Management:** Automatically provisions, scales, and terminates EC2 instances based on Pod requirements (similar to Karpenter architecture), eliminating the need to manually manage Node Groups.
- **Automated Security Patching:** Handles OS-level updates, Kubelet upgrades, and security patching across nodes using zero-downtime rolling strategy.
- **Storage & Networking Integration:** Automates EBS CSI driver lifecycle, EBS volume attachments, and VPC network interfaces.

## What Happens When Enabling vs Disabling EKS Auto Mode?

| Mode State | Behavior & Operational Impact | Recommended Use Case |
|---|---|---|
| **Enable** | AWS fully automates Node lifecycle. No need for Managed Node Groups or Fargate profiles. Compute capability adapts dynamically to workload demands. | New projects, teams seeking reduced operational overhead, and applications focused purely on container deployment. |
| **Disable** | Cluster reverts to standard custom/self-managed mode. You are responsible for instance sizing, Karpenter/CAS configurations, AMI upgrades, and ASG tuning. | Enterprise environments requiring highly customized kernel parameters, custom OS images, or non-standard networking/storage topologies. |

Note: EKS Auto Mode can be enabled during cluster creation or upgraded on existing clusters while co-existing with standard Managed Node Groups if required.

## 2. Authentication Modes: EKS API vs API and ConfigMap

Historically, EKS relied exclusively on an in-cluster configuration map named `aws-auth ConfigMap` to map AWS IAM identities to Kubernetes RBAC roles. AWS introduced **EKS Access Entries** to enable native identity management directly via EKS APIs.

| Authentication Mode | Architecture & Mechanism | Pros & Cons |
|---|---|---|
| **EKS API** Recommended / Modern | Manages cluster authentication directly via AWS IAM APIs using `Access Entries` without altering in-cluster resources. | **Pros:** Declarative management via IaC (Terraform/CloudFormation), full CloudTrail auditability, no risk of YAML formatting corruption. **Cons:** Requires updating legacy toolchains. |
| **API and ConfigMap** Hybrid / Transition | Supports both mechanisms simultaneously. EKS checks Access Entries first; if no match is found, it falls back to `aws-auth ConfigMap`. | **Pros:** Enables seamless zero-downtime migration from legacy setups. **Cons:** Dual management sources increase auditing complexity. |
| **ConfigMap Only** Legacy | Relies entirely on editing the `aws-auth` ConfigMap in the `kube-system` namespace. | **Cons:** High operational risk (formatting errors can lock admins out), difficult to manage declaratively at scale. |

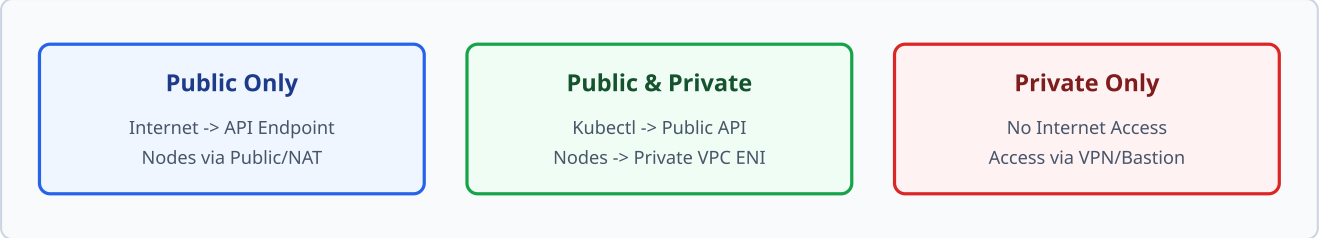
## Key Operational Differences & Migration Strategy

The fundamental difference resides in the **Single Source of Truth** for access control: With **EKS API**, AWS IAM acts as the centralized authority via API calls like `CreateAccessEntry`, removing the need for cluster-level access to grant permissions. Conversely, **ConfigMap** requires write permission into the cluster control plane, where syntax errors can result in immediate cluster lockout.

> **AWS Best Practice Recommendation:**
> AWS recommends migrating fully to the **EKS API** authentication mode and deprecating the in-cluster `aws-auth ConfigMap` to enhance security, governance, and Infrastructure-as-Code automation.

## 3. Cluster Endpoint Access Modes

Every EKS cluster features a Kubernetes API Server Endpoint used by `kubectl` and Worker Nodes to communicate with the Control Plane. AWS provides 3 access strategies to balance security and accessibility:




| Access Mode | API Endpoint Location | Node-to-Control Plane Traffic | Operator Access (Kubectl) |
|---|---|---|---|
| **Public** | Accessible on the public internet. | Routes over the internet (requires NAT Gateway or Public IPs). | Accessible from anywhere (can be restricted via IP CIDR blocks). |
| **Public and Private** Most Popular | Exposed publicly and privately inside VPC. | Routes internally via managed VPC Endpoint ENIs (stays on AWS network). | Accessible directly over the internet without requiring a VPN. |
| **Private Only** Highest Security | Accessible **only** within cluster VPC. | Routes entirely inside the private VPC network. | Requires private VPC presence (via AWS Client VPN, DirectConnect, or Bastion Host). |

### How to Access the Cluster API Server:

- **Public Modes:** Execute `aws eks update-kubeconfig --name cluster_name` and connect directly over standard HTTPS endpoints.
- **Private Mode:** Establish connection to the VPC first (via **AWS Client VPN**, DirectConnect, or SSM Session Manager through a **Bastion Host**), then run `kubectl` commands.

## 4. IP Address Calculation for EKS Worker Nodes & Pods

IP address capacity planning in EKS depends directly on the selected **CNI Plugin**. By default, EKS utilizes the **AWS VPC CNI**, which assigns real VPC IP addresses from your Subnet directly to every Pod, unlike overlay networks (e.g., Calico).

> **Core IP Sizing Formula (AWS VPC CNI):**
> `Total IPs Required = Number of Nodes + (Number of Nodes × Max Pods per Node) + Overhead IPs`
>
> Maximum Pods per node is governed by the EC2 instance type's Elastic Network Interface (ENI) count and secondary IP capacity.

### IP Calculation Breakdown by EC2 Instance Type:

| EC2 Instance Type | Max ENIs | IPs per ENI | Max Pods Capacity per Node | Total IPs Needed for 10 Nodes |
|---|---|---|---|---|
| `t3.medium` | 3 | 6 | (3 × (6 - 1)) + 2 = **17 Pods** | 10 Nodes + (10 × 17) = **180 IPs** |
| `m5.xlarge` | 4 | 15 | (4 × (15 - 1)) + 2 = **58 Pods** | 10 Nodes + (10 × 58) = **590 IPs** |
| `c5.4xlarge` | 8 | 30 | (8 × (30 - 1)) + 2 = **234 Pods** | 10 Nodes + (10 × 234) = **2,350 IPs** |

> **Critical Warning: VPC Subnet IP Exhaustion**
> Because AWS VPC CNI pre-allocates IP pools (Warm ENI Target), subnets can run out of IPs quickly. **Mitigation Strategies:** Enable *Custom Networking* to assign a secondary non-routable CIDR (e.g., `100.64.0.0/16`) dedicated to Pods, or enable *Prefix Delegation* to increase IP density per ENI by 16x.

### Alternative Mode: Overlay Networks (e.g., Calico CNI)

When using Calico in Overlay mode, each Worker Node consumes only **1 single IP** from the AWS VPC subnet, while Pods are allocated private virtual IPs that do not consume VPC addresses.